

Criterios/Lineamientos Generales

Motivación

La idea de este documento es comunicarles un criterio general que siempre va a aplicar a cualquier examen, a fin que todos sepan qué esperar. Obviamente, luego cada examen en particular, y cada ejercicio en particular tienen sus propios criterios, pero los que acá listamos son criterios generales que siempre aplican (es decir, al momento de discutir criterios estos ya están fijos). Esto es también para que lean y sepan qué cosas deben hacer sí o sí (o no deben hacer en absoluto).

Cuestiones generales

- En cada ejercicio que pide dar la complejidad y justificarla, si no está la complejidad, se restará (B-). Si no está correctamente justificada, está simplemente mal, se tomará equivalente a que no esté (es decir, B-).
- Si en un ejercicio se plantea que se debe utilizar una técnica y se decidiera no aplicarla, el ejercicio estará automáticamente Mal (M). "Maquillar" para que parezca que se está utilizando la técnica definida, no se va a tomar como correcto tampoco.
- Si en un ejercicio no se indica la técnica a utilizar de forma explícita, y se utiliza otra a la esperada, lo cual termine en una complejidad peor a la óptima, se restará notoriamente. Cuánto corresponda a esto será evaluado particularmente. Generalmente, darles la técnica es una ayuda para ustedes para que no tengan que pensar además qué utilizar. Cuando no la indicamos, en la enorme mayoría de los casos es porque consideramos que es "sencillo" identificar esto.
- En todos los ejercicios que se deba implementar una solución, debe implementarse con pseudocódigo. No es necesario que sea estrictamente Python, pero no puede ser prosa. En cualquier caso que esto suceda, se considerará como no hecho el código. (Esto no aplica a Programación Lineal, que no se pide una implementación, se pide siempre el modelo).
- Si un ejercicio se resuelve con una complejidad peor a la óptima/más eficiente dadas las características del problema, se restará notoriamente. Cuánto corresponda a esto será evaluado particularmente según el ejercicio y la diferencia entre la complejidad lograda y la mejor. Por supuesto, si se pide implementar con una técnica determinada, será la mejor complejidad lograda con dicha técnica.
- En varios ejercicios pueden implicar tener que trabajar con Grafos. Las primitivas del grafo son las usuales:

- Obtener vértices y/o poder iterar los vértices simplemente (i.e. `for v in grafo`)
- Existe un vértice con X id (o X vértice se encuentra en el grafo)
- Detectar si 2 vértices son adyacentes entre sí (si es dirigido, en el sentido planteado)
- Obtener el peso de la unión entre 2 vértices (si existiera)
- Obtener todos los adyacentes a un vértice
- Agregar o sacar vértices o aristas

División y Conquista

- La justificación de la complejidad va de la mano de la aplicación del Teorema Maestro. Para esto, la ecuación de recurrencia correspondiente al algoritmo debe estar presente. Es decir, **primero debe plantearse la ecuación de recurrencia correspondiente al tiempo que consume el algoritmo**, y luego resolverla utilizando el teorema. En caso de no aplicar, o caer en una respuesta que indique un error conceptual, podría llegar a ser B=. Si la ecuación de recurrencia no se encuentra o no se indica en el correcto desarrollo (se pone la ecuación posterior al desarrollo, lo cual es un error conceptual), eso será seguramente B-.
- Si se aplica algo que no es realmente división y conquista, el ejercicio estará mal. Esto se puede ver simplemente en la ecuación de recurrencia del algoritmo. Si esta no tiene la forma de una ecuación de D&C, es que no lo es. Si tratan de forzarla, sólo se van a estar generando (más) problemas a ustedes mismos.
- En los casos donde no se pueda aplicar el Teorema Maestro y se busque forzar su uso, será un caso particular del primer punto, llegando a ser B=.

Greedy

- En todo ejercicio de implementar un algoritmo greedy para resolver X problema, pedimos la correspondiente justificación de por qué se trata de un algoritmo greedy. Decir "es un algoritmo greedy porque es greedy" (no justifica); "es un algoritmo greedy porque busca que una sucesión de óptimos locales nos lleve al óptimo global" (justificación genérica que no explica qué son esos óptimos), y cualquier otra versión, serán tomadas como incorrectas (B-). Para enunciar correctamente que un algoritmo es greedy se debe poder enunciar su regla greedy y el óptimo local al cual apunta. Si no se puede(n) enunciar, seguramente el algoritmo no es greedy.
- Si se pide una solución óptima, y se da una solución no óptima, se restará. Generalmente será B=, pero puede ser un poco más o un poco menos dependiendo del ejercicio en cuestión.
- Si un ejercicio pide definir si una solución es óptima, en caso de serlo se puede justificar con una breve explicación (no es necesario una demostración formal,

como sí lo es en el TP), en caso de no serlo, sea una justificación o un contraejemplo. Si no se justifica, o la justificación es incorrecta/incompleta, se restará (B-).

- Sí definen que usan una determinada regla greedy, pero no la aplican al estado de la iteración/paso actual, sino al original (y esto no es correcto) entonces no están buscando un óptimo local, por lo que no corresponde realmente a un algoritmo greedy, lo más probable es que no esté aprobable (depende del problema, y del algoritmo). Esto suele darse porque conviene buscar el máximo/mínimo de algo en cada iteración, entonces ordenan inicialmente, cuando es un caso (o algoritmo propuesto) donde la situación de cada elemento se ve modificada en cada iteración, por lo que el orden inicial no sigue valiendo. Podemos ver ejemplos de esto como el algoritmo de Dijkstra, o el de Huffman, mientras que uno donde no se altera la situación de los elementos en cada iteración (y ahí sí conviene ordenar) es el caso de del algoritmo de Kruskal, o bien el que vimos para Scheduling.

Es decir, si en un ejercicio se pide implementar un algoritmo greedy y se pregunta por su complejidad, si es óptimo y por qué es greedy, y se obvian estas 3 preguntas, el ejercicio estará como mucho para Regular (R).

Backtracking

- Si se resuelve por Fuerza Bruta, será Regular (**R**) como mucho. Esto puede ser por una falta completa de podas, o peor aún: calcular todas las soluciones posibles, conservarlas en memoria y luego analizar la mejor solución (esto, al consumir espacio exponencial innecesariamente, será tomado como Mal (**M**)).
- Si se requiere la resolución de un problema por Backtracking, pero deciden aplicar una reducción para resolver otro problema por backtracking (por ejemplo, si se pide de Vertex Cover y reducen a Independent Set, porque ya vimos cómo resolver este último problema por Backtracking en clase), se restará un tanto (**B-** o **B=** según el caso).
- Un error usual (que esperamos erradicar con RPL y que al caer en este, no resuelvan nada) es que actualicen el valor de una variable local esperando que eso se vea por fuera. Eso es un error grave, que no funciona de ninguna forma. Cometer este error implicará que el ejercicio esté regular. Si bien nos parece obvio a estas alturas de la carrera, la solución nunca será utilizar variables globales (hay formas más sutiles de insultar a nuestras madres que hacernos leer eso).
- Usar algo externo que les resuelva temas combinatorios no será tomado como válido y anulará el ejercicio por completo (**M**), dado que demuestran no entender cómo se debe realizar backtracking (literalmente hacen fuerza bruta).
- Un error común es que, al determinar que la solución actual es mejor que la óptima calculada hasta ahora, se devuelve directamente la solución parcial, generando que trabajemos siempre con la misma instancia de solución (entonces, si luego

borramos algo de “la solución actual” para probar otro camino, estamos modificando también el óptimo, lo cual probablemente termine con que la solución óptima tenga cualquier cosa, incluso esté vacía). Esto generalmente será penado con **B-**. También, en caso que se metan copias por doquier, también será penalizado con lo mismo, ya que implica no saber qué se está haciendo (además de un consumo innecesario de memoria).

- Otro error común es devolver algo inmediatamente cuando notamos una mejoría a la solución óptima ya calculada, sin terminar de analizar la rama. Puede ser correcto ya actualizar el óptimo obtenido, pero no decidir terminar si aún se puede mejorar aún más (si no es el caso, entonces sí sería correcto). En caso de tener un error así, generalmente se restaría **B-**, aunque eventualmente según el ejercicio podría llegar a ser **B=**.
- Otro error común es agregar hacks en vez de “programar bien”. Puede ser del estilo hacer parametro[:] = algo, o bien hacer funciones anidadas para evitar pasar un parámetro, o similares. Cualquier cosa que se vea como “código choto” (o bueno, como se diría más formalmente, lleno de “code smells”) restará significativamente. Dependiendo del caso puede ser **B-**, **B=**, o si el código está completamente lleno de esto directamente **R** (ya que demuestra no saber cómo programar algo por backtracking de forma correcta y mantenible). Por ejemplo, el caso de tener funciones anidadas solamente por no saber estructurar el código será puesto como mucho como **R**.
- Sobre “entendimiento del problema”, se espera que se entienda el problema que se está tratando de resolver y no simplemente poner un esquema general de BT. Un error común, por ejemplo, es resolver un problema de optimización de mínimo al que vamos construyendo desde abajo, y no cortar cuando ya se tiene solución, y se siguen agregando elementos. Caso de ejemplo: Vertex Cover (vamos construyendo el VC, si en algún momento tenemos un conjunto que ya es VC, no tiene sentido seguir viendo si “agregamos más vértices”, porque solo vamos a estar empeorando lo actual, y aquí la poda es evidente si entendemos el problema). Hay otros casos que vamos viendo en clase, pero el punto es que hayan cortes que muestren un entendimiento del problema y no una automatización de lo hecho, especialmente cuando el costo de hacer esto es casi nulo (no es que hay que agregar 15 funciones extra para hacer una optimización, es simplemente darse cuenta que de aquí en más solo empeoramos).

Programación Dinámica

- Antes de proceder con la implementación en código, es necesario que esté la ecuación de recurrencia. Si no está la ecuación de recurrencia, es difícil entender si el código está bien o mal, por lo que se restará sustancialmente (**B=** o **R**, según el caso, pero probablemente sea **R**).

- Si se aplica "un algoritmo greedy pero con un arreglo", no es programación dinámica, por lo que estará mal el ejercicio. Un ejemplo claro es el ejercicio 11 de la guía.
- Si realizan la reconstrucción de la solución al mismo tiempo que van calculando los óptimos, estarán cayendo en un doble error. Uno, perjudica necesariamente la complejidad (si lo hacen correctamente, sino no reconstruyen correctamente por reutilizar listas, por ejemplo), y que conceptualmente son dos cosas diferentes. Este error seguramente caiga en que el ejercicio esté regular, o B= como mejor nota (dependiendo del criterio que se adopte para el problema particular).
- También sobre la reconstrucción, en general esperamos que el código sea correcto, y por lo tanto en caso de implementar la reconstrucción esperamos que lo hagan en otra función por separado para tener las cosas correspondientemente separadas (en general, no hacer esto y tener todo el código junto restará B-).
- En caso de no implementar la reconstrucción cuando se pida que se haga, restará sustancialmente (B=).
- Si se escribe una ecuación de recurrencia del estilo $Opt[i] = \min(Opt[i], \dots)$, donde el óptimo depende de sí mismo, se restará significativamente (B=) dado que se trata de un error conceptual importante, donde no se diferencia entre lo que es una ecuación de recurrencia de luego cómo escribimos cosas en el código que puede escribirse de diversas maneras. El código es consecuencia de la ecuación de recurrencia, no al revés.

Programación Lineal

- En todos los casos, lo que se pide es un modelo de programación lineal. No código, y definitivamente no algo de PuLP (que es simplemente la herramienta que usamos). Todas las circunstancias a resolver deben ser tratadas por el modelo.
- Debe estar claro tanto cuáles son las variables, como qué representan, y de qué tipo son (Continuas, Binarias, Enteras). Antes de utilizar una variable, deben estar definidas esas 2 cosas.
- Aquellas cosas que sean constantes/datos para el modelo, debe aclararse (salvo que sea obvio) para no confundirse con las variables.
- Las variables deben en todo momento mantener sus nombres. Si se inventan variables sobre la marcha, en el contexto de un modelo lineal, aduciendo "son las mismas de antes" (cosa que el modelo no ve), implica relajar restricciones haciendo que la modelación no sea válida.
- Al definirse restricciones, debe quedar claro qué están restringiendo (para qué son, respecto al problema, qué son, qué relacionan, o lo que corresponda). No hacerlo restará significativamente. La función objetivo a maximizar o minimizar debe estar brevemente explicada (en algunos casos algo muy breve sería suficiente, si por algún motivo del ejercicio y/o resolución implicara ser *un poco más compleja*, entonces que esté la explicación correspondiente). En caso que no se hagan

explicaciones de ningún caso, el ejercicio no estará aprobable (lo más probable es que quede para Regular).

- Cualquier caso de dar un modelo no lineal (multiplicación de variables) anula la resolución.
- Tener restricciones de más, cuando exista una forma de hacerlo con menos, restará (B- o B= según se considere por el ejercicio). Como ejemplo de esto, es lo que vemos del caso de Independent Set. No hablamos de 1 o 2 restricciones más, sino de una cantidad significativa.

Redes de Flujo

- Si el ejercicio implica modelar un problema con una red de flujo para luego calcularle, valga la redundancia, el flujo, y así obtener una solución al problema original, se espera que hagan:
 - Una explicación de la modelación (con palabras y/o dibujos, para que quede claro). No hacer esto restará un poco de por sí, pero considerar que no dejarnos claro cómo modelan nos puede hacer no entender si la solución es correcta. Recuerden que no podemos leerles la mente.
 - El código que ustedes deben implementar no recibirá un grafo modelado como indicaron antes. Ustedes recibirán la información del problema, y deben ustedes transformar esto en una red de flujo (léase, el código de lo explicado arriba).
 - El código del algoritmo de Ford-Fulkerson lo pueden asumir ya implementado. No es necesario que hagan algo sobre esto. El algoritmo devolverá un diccionario con el flujo máximo por arista. Es importante notar que la red ya debe ser de flujo. Es decir, si hay que resolver algún impedimento (aristas antiparalelas, bucles, más de una fuente o sumidero) debe realizarse antes, y eso debe considerarse en el siguiente paso.
 - La respuesta del algoritmo que ustedes implementan difícilmente sea directamente el flujo sobre la red definida. Seguramente deban transformar eso en una solución correcta. Esto es parte de la solución. Por lo general, no hacer esto implicará restar algo (B-).
 - Recordar que la complejidad debe estar siempre dada por los parámetros de entrada. Por ejemplo, si se da una lista de información y se dice " $O(V \cdot E^2)$ " cuando en el problema originalmente no había un grafo, se considerará incorrecto. Lo mismo sucede si en efecto había un grafo, pero ustedes debieron crear otro a partir de este. Pueden partir, obviamente, de que esa es la complejidad inicialmente, y a partir de ello deducir la complejidad en función de los parámetros de entrada (sean del tipo que fueren).
- Si el ejercicio implica modificar el algoritmo de Ford-Fulkerson por el motivo que fuere, en este caso, por supuesto, no pueden asumir dicho algoritmo como implementado (sería imposible de plantear modificarlo). Para esto, por supuesto,

deben saber cómo se implementa dicho algoritmo. En cualquier caso, cualquier parte, función auxiliar que no sea necesaria modificar, pueden asumirla implementada.

Reducciones y Clases de Complejidad

- Si un ejercicio pide demostrar que X es un problema NP-Completo, el primer paso debe ser demostrar que se trata de un problema que está en NP. Para esto, deben escribir el código del validador, y luego indicar y justificar su complejidad, a fin de demostrar que se trata de un algoritmo polinomial. Si esto no está, se restará (entre B^- y $B=$, en función del ejercicio). Si no se justifica la complejidad (es decir, se da un validador y directamente se concluye que es un problema en NP), se tomará como incompleto y se restará (B^-). Si el validador se hace de una forma no óptima, se tomará como incorrecto también (por ejemplo, si se puede implementar en tiempo lineal y se implementa en tiempo cuadrático).
- La reducción a hacer debe ser clara. Recomendamos fuertemente poner algún ejemplo que nos permita entender qué es lo que se está haciendo. Traten que no sea el ejemplo más simple (ej: un grafo con 2 vértices) que pueda no dejar claro qué hacen en cada caso. Es de su interés que entendamos bien a partir de la explicación y ejemplos, porque si no queda claro, va a estar incompleta la definición. Si falta algo en el texto, o es ambiguo, pero luego en un ejemplo lo dejan claro, entonces no hay problema. No debe quedar de la explicación algo librado a nuestra imaginación y "que es obvio", porque no podemos saber qué es lo que están pensando.
- Un error muy común es olvidarse de que hay que trabajar con los problemas en su versión de decisión. Asegúrense de definir el problema de decisión con claridad, y recordar que la caja negra devuelve `True` o `False`. Asumir que se devuelve un resultado (ej: un conjunto de vértices) para poder manipularlo en todos los casos sería un error, que reste significativamente ($B=$, R o M según se considere).
- En cualquier caso que la reducción utilizada no fuera polinomial, el ejercicio por completo estará mal.
- Al realizar la reducción, debe *demostrarse* que es correcta. Es decir, si reducimos el problema Y al problema X (para demostrar que $Y \leq_p X$), entonces se debe explicar el "sí sólo sí" o "ida y vuelta":
 - Si dada una instancia arbitraria del problema Y en el cual la respuesta sea *true*, ¿la reducción realizada termina devolviendo ese mismo resultado?
 - Si dada una instancia arbitraria del problema Y , si realizando la reducción planteada al problema X , este devuelve *true*, ¿corresponde esto también al problema Y ?
 - Considerar los puntos anteriores como los usuales, si por X motivo a reducción implica devolver lo contrario a lo que diga la reducción, entonces

plantear lo mismo pero al revés, lo mismo si se hace más de una invocación, etc...

- **En caso que no haya explicación de este estilo, se restará notoriamente, según el caso.** Lo más usual sería B=. Es decir, si el verificador no se hace de forma correcta, o no se justifica su complejidad, y luego no se justifica por qué la reducción sería correcta, el ejercicio no estaría aprobable.

Aproximaciones

- Si el ejercicio pide implementar una aproximación a un problema, y se termina implementando una ∞ -aproximación (es decir, existe la posibilidad que la solución del algoritmo sea infinitas veces peor que la óptima, sea por de más o por de menos), entonces el ejercicio quedará anulado por completo.
- Cuando se pida una demostración, debe ser hecho de forma correcta, tal como hacemos en otras partes de la materia.

Algoritmos Randomizados

[En desarrollo]

Definición de notas

La nota de los temas/exámenes se calcula a partir de tener al menos 5 temas aprobados, considerando:

- Que cada ejercicio suma puntos según la nota: NH (no hecho) o M: 0 puntos; R: 1 punto, B=: 2 puntos; B-: 3 puntos; B: 4 puntos.
- Aplicamos la fórmula: $0.37 * \text{puntos} + 0.3$.

Por ejemplo, si tenemos como notas: B, B-, R, B, B=, M, NH, sumariamos 15 puntos, obteniendo de nota 5,85. Si tenemos B-, B-, B, B, R, B, B-, sumariamos 22 puntos, obteniendo de nota 8,44. Notar que si tenemos todos los ejercicios B, tendríamos 10,66 de nota.

Es decir, podemos obtener la nota numérica con:

```
Python
PUNTOS = {"B": 4, "B-": 3, "B=": 2, "R": 1}

def nota_numerica(notas): # lista/tupla de notas
    puntos = 0
```

```
for nota in notas:
    if nota in PUNTOS:
        puntos += PUNTOS[nota]
return round(0.37 * puntos + 0.3, 2)
```